

# **CROWD MONITORING AND SAFETY SYSTEM**

A Mini Project Report 1 submitted in partial fulfillment of the  
requirements for the award of degree of

**BACHELOR OF TECHNOLOGY**

**IN**

**ELECTRONICS & COMMUNICATIONS ENGINEERING**

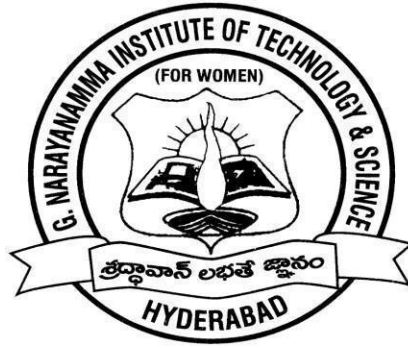
Submitted By

|                         |                   |
|-------------------------|-------------------|
| <b>M. TWINKLE DURGA</b> | <b>23251A04A8</b> |
| <b>N. PRAVEENA</b>      | <b>23251A04B2</b> |
| <b>S. HANISHA</b>       | <b>23251A1228</b> |
| <b>B. RUCHITHA</b>      | <b>23251A1208</b> |

Under the esteemed guidance of

**Mrs. P. Madhuri**

Assistant Professor, ECE, GNITS (FOR WOMEN).



**DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGG. G.  
NARAYANAMMA INSTITUTE OF TECHNOLOGY & SCIENCE**

(For Women)

**(AUTONOMOUS)**

**(Accredited by NBA & NAAC)**

2024-2025

**DEPT. OF ELECTRONICS & COMMUNICATION ENGG**

**CERTIFICATE**

THIS IS TO CERTIFY THAT THE MINI PROJECT 1 ENTITLED

**CROWD MONITORING AND SAFETY SYSTEM**

IS THE BONAFIDE WORK OF

|                   |            |
|-------------------|------------|
| M. TWINKLE DURGA  | 23251A04A8 |
| N. PRAVEENA       | 23251A04B2 |
| S. HANISHA        | 23251A1228 |
| B. RUCHITHA REDDY | 23251A1208 |

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE  
AWARD OF DEGREE OF BACHELOR OF TECHNOLOGY IN ELECTRONICS AND  
COMMUNICATION ENGINEERING DURING THE YEAR 2024-2025

Mrs. P. Madhuri  
Assistant Professor, ECE

Dr. K. Ragini  
PROF & HOD, ECE

## ACKNOWLEDGEMENTS

It is our privilege to express the feelings of gratitude to several people who helped us in completing the project.

We take this opportunity to express our heartfelt gratitude to our esteemed guides, **Mrs. P. Madhuri**, Assistant Professor, ECE department and **Mrs. V. Usha**, Assistant Professor, IT department, GNITS for her constant guidance, encouragement, and immense motivation.

We are thankful to our Mini Project coordinators, **Dr. B. Pavani**, Assistant Professor, ECE department and **Mrs. M. Sridevi**, Assistant Professor, IT department, GNITS for her valuable advice and encouragement.

We express our heartfelt gratitude to **Dr. K. Ragini**, Head of the department, ECE department, **Dr. S. Ramcharan**, Head of the department, IT department GNITS for their constant guidance and moral support.

We would like to express our sincere thanks to **Dr. K. Ramesh Reddy**, Principal, GNITS for providing working facilities in the college.

Finally, we would like to thank all the faculty and staff of ECE department who helped us during our project and also our parents and friends for their cooperation in completing the project.

Twinkle Durga 23251A04A8

N Praveena 23251A04B2

S Hanisha 23251A1228

B Ruchitha 23251A1208

# CONTENTS

| S. No. | Topic                        | Page No. |
|--------|------------------------------|----------|
|        | Abstract                     | i        |
|        | List of Figures              | ii       |
|        | List of Acronyms             | iii      |
| 1.     | Introduction                 | 1        |
|        | 1.1 Problem Statement        | 1        |
|        | 1.2 Literature Survey        | 1        |
|        | 1.3 Objectives               | 2        |
|        | 1.4 Methodology              | 3        |
| 2.     | System Architecture          | 6        |
|        | 2.1 Hardware Components      | 6        |
|        | 2.1.1 ESP32 Microcontroller  | 6        |
|        | 2.1.2 ESP32 Camera Module    | 7        |
|        | 2.1.3 ESP32 Camera Micro-USB | 8        |
|        | 2.1.4 DHT11 Sensor           | 9        |
|        | 2.1.5 LCD Display            | 10       |
|        | 2.1.6 Jumper Wires           | 10       |
|        | 2.1.7 PCB                    | 10       |
|        | 2.2 Software Components      | 10       |
|        | 2.2.1 Arduino IDE            | 11       |
|        | 2.2.2 Python(OpenCV,YOLO)    | 11       |
|        | 2.2.3 Twilio API             | 12       |
| 3.     | Implementation               | 13       |
|        | 3.1 Hardware Implementation  | 14       |
|        | 3.2 Software Implementation  | 15       |
|        | 3.3 Performance Check        | 16       |

|    |                            |    |
|----|----------------------------|----|
| 4. | Results                    | 18 |
| 5. | Advantages & Disadvantages | 21 |
| 6. | Applications               | 22 |
| 7. | Conclusions                | 23 |
| 8. | Future Scope               | 24 |
|    | References                 | 25 |

## **ABSTRACT**

Nowadays, public security in many areas has become even more important, particularly in temples, public gatherings. Our project is based on IoT and computer vision to identify, track, and act on potentially risky crowd conditions in real time. The system employs the ESP32-CAM module integrated with OpenCV and the YOLO (You Only Look Once) object detection algorithm implemented in Python for real-time estimation of crowd density. This enables precise identification of individuals within the field of view of the camera and provides real-time data for further processing. Together, an ESP32 microcontroller is coupled with a DHT11 temperature and humidity sensor to observe environmental conditions, which play an important role in avoiding overheating or suffocation hazards in crowded spaces. By integrating crowd analysis and environmental sensing, the solution presented provides an advanced safety mechanism, with the possibility of automated warnings and alerts upon violation of crucial limits. This improves situational understanding and enables informed decision-making in real time, advancing public safety and event management in general.

## **LIST OF FIGURES**

| <b>S. No.</b> | <b>Fig. No.</b> | <b>Name of the Fig.</b> | <b>Page No.</b> |
|---------------|-----------------|-------------------------|-----------------|
| 1.            | 1.1             | Block Diagram           | 4               |
| 2.            | 1.2             | Flowchart               | 5               |
| 3.            | 2.1.            | ESP32 Pin Diagram       | 8               |
| 4.            | 2.2             | ESP32-CAM Pin Diagram   | 9               |
| 5.            | 2.3             | ESP32-CAM Micro USB     | 10              |
| 6.            | 2.4             | DHT11 Sensor            | 11              |
| 7.            | 2.5             | LCD Display             | 12              |
| 8.            | 2.6             | Python and OpenCV       | 14              |
| 9.            | 4.1             | Crowd Detection         | 21              |
| 10.           | 4.2             | Temperature reading     | 22              |
| 11.           | 4.3             | SMS Alert Message       | 22              |
| 12.           | 4.4             | Alert Message on GMAIL  | 23              |
| 13.           | 4.5             | Crowd Monitoring System | 23              |

## LIST OF ACRONYMS

| S.No. | Abbreviations | Full Forms                                  |
|-------|---------------|---------------------------------------------|
| 1.    | IDE           | Integrated Development Environment          |
| 2.    | IR            | Infra Red                                   |
| 3.    | DHT           | Digital Humidity and Temperature            |
| 4.    | IoT           | Internet of Things                          |
| 5.    | YOLO          | You Only Look Once                          |
| 6.    | SMS           | Short Message Service                       |
| 7.    | API           | Application Programming Interface           |
| 8.    | ESP           | Espressif Systems Product                   |
| 9.    | GPIO          | General Purpose Input/Output                |
| 10.   | OPENCV        | Open Source Computer Vision                 |
| 11.   | USB           | Universal Serial Bus                        |
| 12.   | LCD           | Liquid Crystal Display                      |
| 13.   | PCB           | Printed Circuit Board                       |
| 14.   | UART          | Universal Asynchronous Receiver-Transmitter |
| 15.   | SMTP          | Simple Mail Transfer Protocol               |
| 16.   | FPS           | Frames Per Second                           |
| 17.   | ADC           | Analog-to-Digital Converter                 |
| 18.   | DAC           | Digital-to-Analog Converter                 |
| 19.   | TX            | Transmit                                    |
| 20.   | RX            | Receive                                     |



# 1. INTRODUCTION

## 1.1 Problem Statement

Overcrowding in public spaces such as temples, marketplaces, college events, and festivals poses serious safety risks, including discomfort, panic, suffocation, and stampedes. Existing crowd monitoring systems are often manual, sensor-limited, or costly, making them inefficient and impractical for dynamic environments. Moreover, most current systems lack real-time environmental awareness, which is essential for detecting potential hazards. There is a growing need for an affordable, adaptable, and automated solution that can not only monitor crowd density accurately but also ensure public safety through timely alerts and environmental sensing.

## 1.2 Literature Survey

To design a system that effectively monitors crowd density and ensures public safety, we conducted a detailed literature review of existing methods and research in the domain of crowd management. Various technologies have been proposed and implemented over the years, each with their own advantages and limitations. Our focus was to understand what's already been tried and where our solution could improve upon it.

This paper proposes a crowd monitoring system using Arduino UNO R4, IR sensors, and Arduino Cloud for real-time updates. It is affordable and IoT-based but works best only in small, controlled environments with static sensor placement[1].

It uses machine learning algorithms to monitor crowd activity in educational institutions. However, it's highly dependent on training data quality and requires substantial computing power[2].

This work applies a custom-trained YOLO model to estimate crowd density and service level. Its accuracy depends on training data, and it may not adapt well to different environments[3].

A Raspberry Pi-based system using image processing to monitor people and social distancing. While cost-efficient, it struggles with real-time performance due to hardware limitations[4].

Focuses on transmitting CCTV data to recognize behavioral patterns in crowds. Prioritizes security and data transmission over real-time density estimation[5].

Most systems focus only on detection, with no real-time alerting mechanism like buzzers or SMS for emergencies.

Environmental conditions like temperature and humidity were not integrated, which are critical in enclosed or densely packed areas.

Many solutions rely on high-cost or high-power platforms like Raspberry Pi or IoT Cloud, making them less suitable for budget-conscious deployments.

Sensor-based systems were found to be highly sensitive to placement, limiting scalability and reliability.

Surveillance-focused models emphasized data transmission and behavior, but overlooked live, visual crowd density estimation.

Our Contribution:

Our project addresses these gaps by combining YOLO-based real-time crowd detection, DHT11 environmental sensing, and an automated alert system using a buzzer and optional SMS (Twilio API) — all built using affordable hardware like ESP32-CAM for broader accessibility and practical field deployment.

### **1.3 Objectives**

Overcrowded public spaces such as temples, markets, college events, and festivals can quickly become unsafe without timely detection and intervention. Traditional crowd monitoring methods depend heavily on human supervision or basic sensors, which lack real-time adaptability and accuracy. Many of these systems also ignore environmental conditions that contribute to discomfort and risk during crowd buildup. Our project aims to address these limitations by creating an intelligent, affordable, and scalable system that combines computer vision with environmental sensing to ensure public safety in crowded environments.

#### **1.3.1. To Implement Real-time Crowd Detection, Estimation & Temperature Monitoring**

- Implement a computer vision system using OPENCV, YOLOv8 to detect and count people in live video feeds.
- Integrate the DHT11 sensor to monitor temperature and humidity conditions in real time.
- Identify unsafe environmental conditions that could result from overcrowding, such as heat buildup or reduced ventilation.
- Enable the system to respond not only to visual crowd detection but also to environmental stress indicators.

#### **1.3.2. To Implement Alert and Response Mechanism**

- Develop an automatic alert system using GMAIL to notify authorities when thresholds are exceeded.
- Integrate Twilio API for sending SMS alerts in critical crowd conditions and alert the emergency services by providing them location of the overcrowd..
- Ensure alerts are triggered based on combined visual and environmental data along with location.

## 1.4 Methodology

Our proposed system is designed to monitor crowd density using computer vision and environmental sensors, with real-time alerts triggered for unsafe conditions. The complete system is built around both hardware integration and software logic to ensure affordability, portability, and real-time responsiveness.

### Setting Up the Hardware

#### ESP32-CAM for Live Video Feed

- Component: ESP32-CAM microcontroller with built-in camera.
- Connection: Configured to stream live video over a local IP via Wi-Fi.
- Purpose: Captures real-time frames for crowd detection using computer vision.

#### Environmental Sensors

- Component: DHT11 (for temperature and humidity).
- Connection: Connected to ESP32 GPIO pins.
- Purpose: Monitors environmental conditions that may signal heat buildup in dense areas.

#### Alert Module

- Sending SMS using Twilio API
- Sending an ALERT message through GMAIL

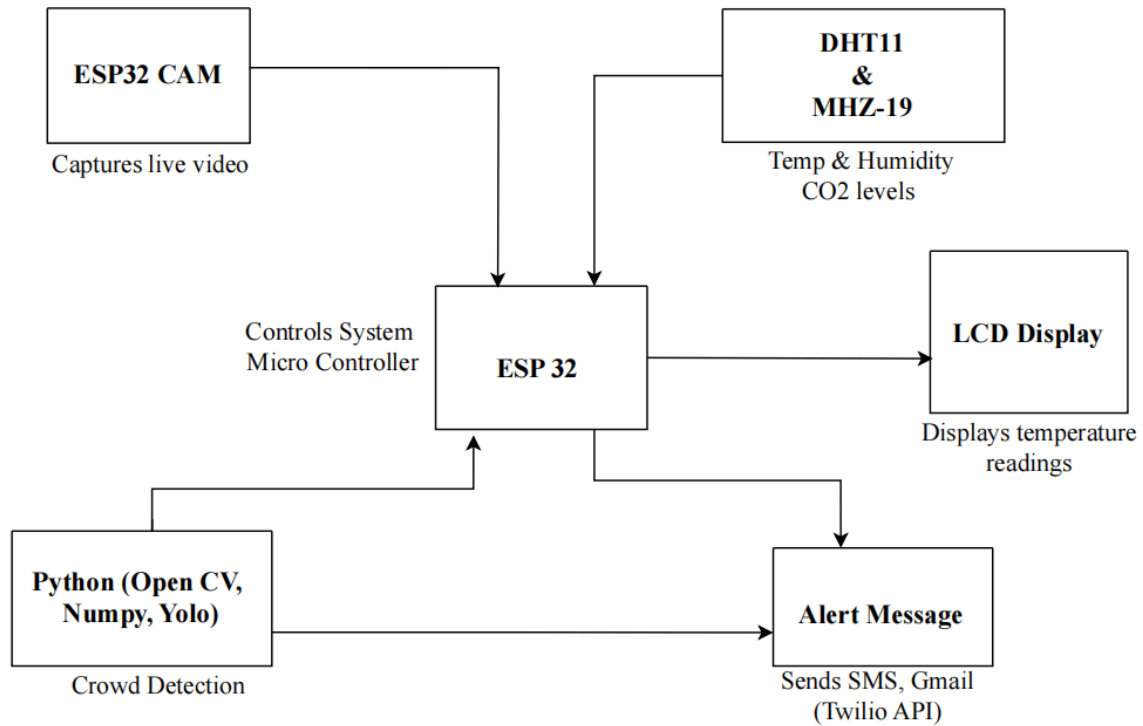


Fig. Block Diagram of Crowd Monitoring System

Fig. 1.1 - Block Diagram

## Programming and Logic (Software Flow)

### Frame Extraction and Headcount

- Tool: Python with OpenCV and YOLOv8 (via Ultralytics).
- Process
  - Captures each video frame and runs a YOLOv8 model to detect people.
  - Extracts bounding boxes and counts them.

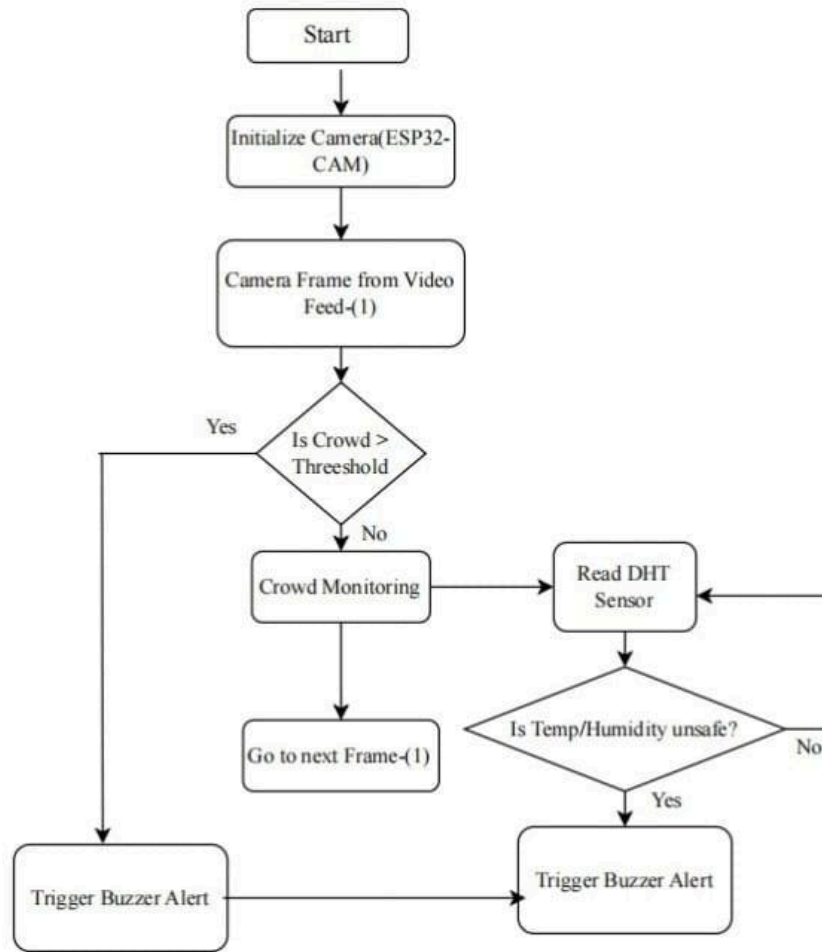


Fig. 1.2 - Flowchart

#### Environmental Data Processing

- Tool: Arduino IDE
- Input: Temperature and humidity from DHT11.
- Logic
  - Reads data via serial.
  - Compared to safety thresholds.
  - Combined with crowd count for risk evaluation.

#### Alert Triggering

- When crowd count exceeds limit or temp > threshold:
  - Email Alert will be sent.
  - Sends SMS alerts via Twilio API.

## 2. SYSTEM ARCHITECTURE

### 2.1 Hardware Components

- ESP32 Microcontroller
- ESP32 Camera Module
- ESP32 Camera Micro-USB
- DHT11 SENSOR
- 16\*2 I2C LCD Display
- Jumper Wires
- PCB

#### 2.1.1 ESP32 Microcontroller

ESP32 is a family of low-cost, energy-efficient microcontrollers that integrate both Wi-Fi and Bluetooth capabilities. These chips feature a variety of processing options in both dual-core and single-core variants.

In addition, the ESP32 incorporates components essential for wireless data communication such as built-in antenna switches, power amplifiers, low-noise receivers, filters, and power-management modules.

Typically, the ESP32 is embedded on device-specific printed circuit boards or offered as part of development kits that include a variety of GPIO pins and connectors, with configurations varying by model and manufacturer. The ESP32 was designed by Espressif Systems and is manufactured by TSMC using their 40 nm process. It is a successor to the ESP8266 microcontroller.

The ESP32 typically has 30 to 38 GPIO pins, depending on the model (like ESP32-WROOM-32). Most GPIOs are multipurpose, meaning they can be used for digital I/O, ADC, DAC, PWM, etc.

- ESP32 has around 25 usable digital I/O pins. You can use most GPIOs except GPIO 6 to 11 (they're connected to the flash chip). GPIOs 34 to 39 are input-only.
- It has 18 analog input pins (ADC). The best ones to use, especially when WiFi is on, are GPIOs 32 to 39 (they belong to ADC1).
- There are 2 analog output pins (DAC): GPIO 25 and GPIO 26. These can output real analog voltages from 0 to 3.3V.

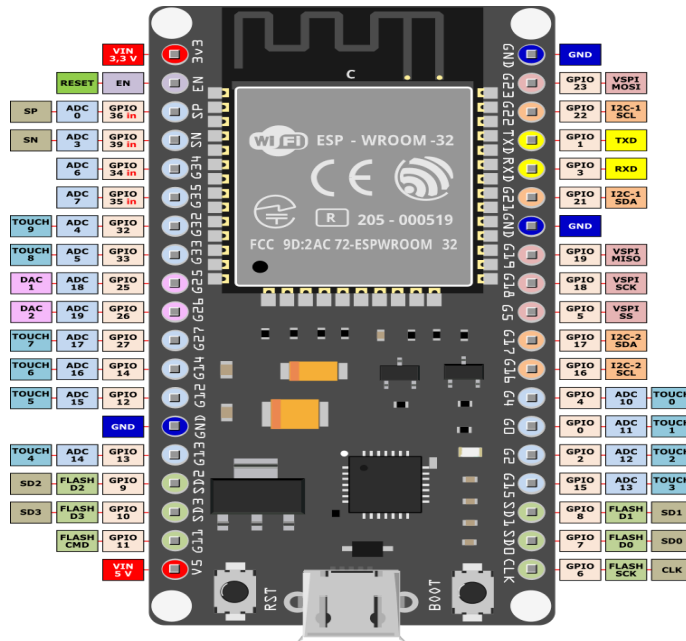


Fig. 2.1 - ESP32 Pin Diagram

### 2.1.2 ESP32 Camera Module

The ESP32-CAM is a compact, low-cost development board that integrates a camera module along with built-in Wi-Fi and Bluetooth. It is commonly used in projects that involve image capture, video streaming, face detection, and wireless camera-based applications.

This board comes with a 2MP OV2640 camera and supports various image resolutions, making it suitable for real-time video transmission over Wi-Fi. It also features a microSD card slot for local data storage. However, unlike standard ESP32 boards, it lacks a built-in USB port, so programming requires an external USB-to-Serial adapter.

Internally, the board uses the ESP32-S chip and includes essential components like voltage regulators and antenna connectors. It offers around 9 usable GPIO pins, but since many are linked to the camera and SD card, pin usage must be planned carefully to avoid conflicts.

GPIO usage details

- GPIO 0: Must be pulled LOW to upload code (flashing mode).
- GPIOs 1 and 3: Used for UART (TX and RX).
- GPIOs 4, 5, 12 to 16: Connected to the camera interface.
- GPIOs 33, 34, 35, 36, 39: May be available for input-only operations.

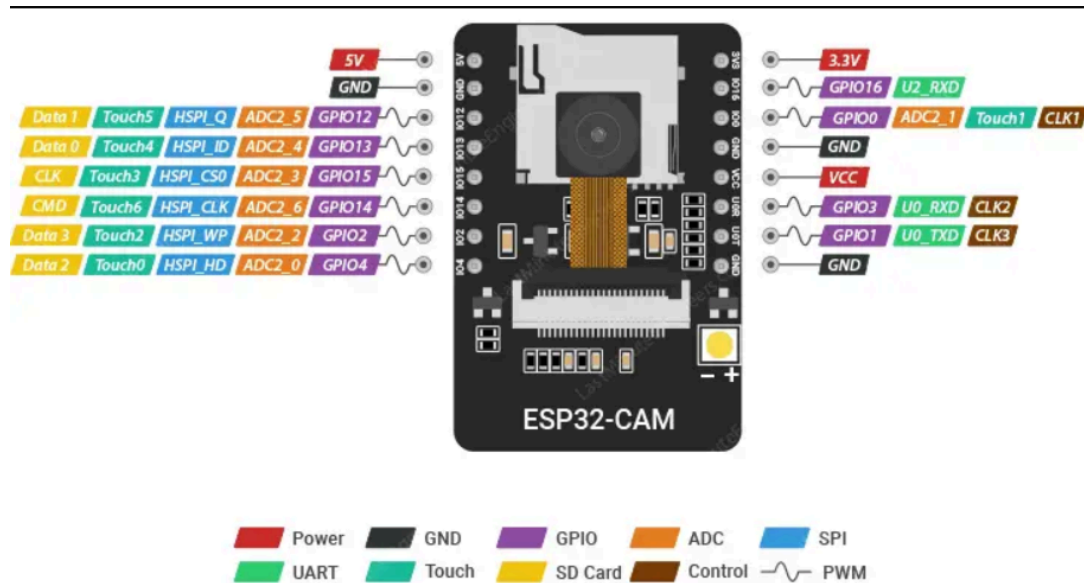


Fig. 2.2 - ESP32-CAM Pin Diagram

### 2.1.3 ESP32 Camera Micro-USB

The ESP32-CAM with Micro-USB is a compact development board that integrates a camera module, Wi-Fi, Bluetooth, and a built-in USB port. This version eliminates the need for an external USB-to-Serial adapter, making programming easier. It's perfect for quick prototyping and IoT camera projects.

It comes equipped with a 2MP OV2640 camera that supports multiple resolutions suitable for image capture and real-time video streaming over Wi-Fi. All core features of the original ESP32-CAM are retained, with the added benefit of easier connectivity through the Micro-USB interface. The board is ideal for wireless surveillance and embedded AI applications.

GPIO usage details

- GPIO 0: Used for flashing; handled automatically through onboard USB circuitry in most versions.
- GPIOs 1 and 3: Reserved for UART communication.
- GPIOs 4, 5, 12 to 16: Connected to the camera interface.
- GPIOs 33, 34, 35, 36, 39: May be available as input-only GPIOs.





Fig. 2.3 - ESP32-CAM MICRO USB

#### 2.1.4 DHT11 Sensor

The DHT11 is a low-cost digital sensor used to measure temperature and humidity. It's widely used in weather monitoring, indoor climate control, and other basic environmental sensing projects. It's simple to use, small in size, and works well with microcontrollers like the ESP32.

This sensor provides digital output, which means it sends temperature and humidity readings directly to the microcontroller without needing any extra calculations. It can measure temperatures from 0°C to 50°C and humidity levels from 20% to 90%. The data is sent over a single data pin, making wiring simple and clean.

The DHT11 operates on 3.3V to 5V power and is easy to interface with most boards. It sends data in regular intervals (usually every 1 second), so it's perfect for projects that need constant monitoring. While it's not the most accurate sensor, it's reliable enough for hobby and student-level projects.

Pin configuration details:

- VCC: Power supply (3.3V or 5V)
- DATA: Sends digital temperature and humidity readings
- GND: Ground connection



Fig. 2.4 - DHT11 Sensor

### 2.1.5 LCD Display

The LCD 16x2 is a beginner-friendly display that shows 2 lines of text with 16 characters each, perfect for showing messages or sensor data. With an I2C module, it needs only 4 connections—VCC, GND, SDA, and SCL—making wiring simple and neat. It runs on 5V and works smoothly with microcontrollers like the ESP32 in real-time projects.



Fig. 2.5 - LCD Display

### 2.1.6 Jumper Wires

Jumper wires are small insulated wires used to connect different parts of a circuit on a breadboard or between modules. They come in three types—male-to-male, male-to-female, and female-to-female—depending on the pin/socket type needed. These wires make prototyping clean and easy without soldering. They're perfect for students and beginners working with microcontrollers like the ESP32.

### 2.1.7 PCB

A PCB is a flat board used to mount and connect electronic components in a neat and permanent way. It has copper tracks that act like wires, carrying signals and power throughout the circuit. PCBs are used in final versions of projects for durability and better performance. They're ideal when moving from a breadboard to a finished, long-lasting setup.

## 2.2 Software Components

- Arduino IDE
- Python
- OpenCV
- YOLO
- Twilio API

### **2.2.1 Arduino IDE**

The Arduino IDE(Integrated Development Environment) is a software application used to write and upload code. It provides a simple, user-friendly interface for programming microcontrollers,making it an accessible platform for beginners.

The Arduino IDE provides a platform to write, compile, and upload Arduino sketches. Key components include the Code Editor, Message Area, Text Console, Toolbar, Menus, Board and Library Management, Debugging Tools, and Sketch Syncing. There are several types: Arduino IDE 1.x (classic and simple), IDE 2.0 (modern with autocompletion and live debugging), Arduino Pro IDE (for advanced users), Cloud Editor (online access and syncing), and PLC IDE (for industrial use with IEC 61131-3 support).

How it Works:

- You write your code in the text editor within the Arduino IDE.
- The IDE compiles the code into a machine-readable format.
- The IDE then uploads the compiled code to your connected ESP32 DEV MODULE board.
- The ESP32 DEV MODULE board executes the uploaded code.

### **2.2.2 PYTHON(OpenCV, YOLO)**

Python, in conjunction with OpenCV and the YOLO algorithm. OpenCV is a library for image and video processing, while YOLO (You Only Look Once) is a real-time object detection algorithm. By leveraging these tools, you can build systems that automatically identify and locate objects within visual data.

OpenCV (Python):

OpenCV is a widely used library that offers a range of functionalities for image and video processing, including image manipulation, feature detection, and object detection. In Python, you can use OpenCV to load images and videos, preprocess them, and visualize results.

YOLO (You Only Look Once):

YOLO is a deep learning algorithm that excels at object detection. It divides the image into a grid and predicts bounding boxes and class probabilities for each grid cell. This allows YOLO to detect multiple objects simultaneously and with high speed.

Python:

Python serves as the programming language to implement the object detection pipeline. You can use libraries like OpenCV and TensorFlow (for loading and using pre-trained YOLO models) to create object detection systems in Python.



Fig. 2.6 - Python and OpenCV

### 2.2.3 Twilio API

Twilio's Programmable Messaging API is a reliable SMS API used for sending and receiving SMS messages. You can use it to send transactional SMS messages with features like managing phone numbers, ensuring deliverability, and handling compliance. The API can be used for various applications, including alerts, notifications, promotions, and verification codes

Reliability:

Twilio's SMS API is known for low latency, high delivery rates, and a large inventory.

### 3. IMPLEMENTATION

To build a real-time crowd monitoring and environmental alert system, the following components and steps are required:

#### Implementation Steps

- **ESP32-CAM Setup**
  - Flash the ESP32-CAM using the Arduino IDE.
  - Configure the code to capture frames and read data from the DHT11 sensor.
  - Establish Wi-Fi connectivity to enable streaming and communication with the backend or cloud server.
- **Sensor and Camera Integration**
  - Connect the DHT11 sensor to the ESP32 GPIO pins (usually D2 or D4).
  - Configure the sensor library in the code to periodically read temperature and humidity values.
  - Set camera parameters such as resolution and brightness to optimize detection quality.
- **Crowd Detection Logic**
  - Frames captured by the ESP32-CAM are analyzed using the Haar cascade classifier in the backend to detect faces or bodies.
  - Count the number of detected entities to estimate crowd density.
  - If the crowd count exceeds a set threshold, trigger alerts via SMS and email.
- **Environmental Monitoring**

The DHT11 sensor reads ambient conditions continuously.

  - If temperature or humidity exceeds predefined limits, alert functions are invoked automatically.
- **Alert System Integration**
  - Twilio API credentials and phone numbers are configured in the Python script.
  - Gmail SMTP credentials are set securely to send emails with sensor and crowd data.
  - Alert frequency is throttled to avoid spamming during repeated threshold breaches.

- Assembly and Testing
  - Mount the ESP32-CAM and DHT11 sensor securely on a board or wall in a target area.
  - Ensure all connections are stable and protected from external damage.
  - Conduct indoor and outdoor tests to verify detection accuracy under varying light, crowd size, and environmental conditions.
  - Fine-tune thresholds and camera angles based on initial testing feedback.

### **3.1 Hardware Implementation**

#### **• Sensor Selection**

For effective crowd monitoring and environmental tracking, sensor selection plays a pivotal role. The DHT11 sensor is utilized for monitoring temperature and humidity, which aids in analyzing human comfort and environment crowd density trends. It offers an adequate range and reliability for indoor environments. For motion or presence detection, camera modules or ultrasonic sensors can be considered, but in our case, we are using a ESP32-CAM module, which provides both video streaming and image capture functionalities. The choice is based on its built-in Wi-Fi capabilities, compact design, cost-efficiency, and compatibility with real-time data processing needs. The Haar cascade algorithm is implemented to detect human presence from the ESP32 camera feed.

#### **• Sensor Mounting**

The ESP32-CAM is mounted in a strategic position that ensures maximum coverage of the target area — typically at an elevated angle, such as the top of a doorway or corner of a room, to capture a wide field of view. The DHT11 sensor is placed in a location free from obstructions and heat sources to ensure accurate environmental readings. Care is taken to avoid reflections or occlusions that could affect the camera's input and the efficiency of the Haar detection algorithm. All components are housed in protective casings to prevent environmental damage, particularly when deployed in semi-open public areas.

#### **• Microcontroller Interfacing**

The ESP32 microcontroller is programmed to handle video input and wirelessly transmit frames for processing. It is interfaced with the DHT11 sensor and connected to Wi-Fi for real-time alerts and remote access. Standard GPIO pins are used to connect the sensor, while the ESP32's built-in UART and I2C communication protocols are leveraged for seamless operation. The Python application receives data from the ESP32 and

processes it using OpenCV and pre-trained Haar cascade classifiers. Additionally, the Arduino is optionally used to assist with LCD display output and sensor-based decisions.

- **Communication and Alert System Integration**

To handle alerts, the system integrates Twilio and Gmail SMTP for sending SMS and email notifications when certain thresholds are exceeded (e.g., high crowd density or temperature alerts). The ESP32 sends alerts through Wi-Fi based on sensor inputs, and the Python application triggers communication protocols accordingly. Proper authentication, API key management, and error handling are implemented to ensure reliable delivery of alerts.

- **Power Supply**

A stable power source is critical for the continuous operation of ESP32, DHT11, and optionally Arduino modules. The system is powered using a 5V regulated power adapter or Li-ion batteries with USB recharge capabilities. Voltage regulators ensure each component receives the required voltage. Care is taken to protect the circuit from power surges or battery discharge through overvoltage protection circuits. In deployments requiring mobility or wireless operation, battery monitoring circuits are added for safety and to avoid downtime.

## **3.2 Software Implementation**

- **Sensor Data Processing**

The system collects environmental data using the DHT11 sensor and image data from the ESP32-CAM module. For the DHT11, raw data (temperature and humidity) is received via digital signals and parsed by the Arduino or ESP32 microcontroller. In the case of camera input, real-time frames are transmitted wirelessly from the ESP32 to a Python script. Image data is processed using OpenCV libraries and Haar cascade classifiers to detect the presence of human faces or bodies. Basic noise reduction and frame skipping are implemented to optimize processing time. Sensor data is periodically logged, and thresholds are defined for triggering alerts.

- **Crowd Detection Algorithm**

The crowd detection logic is handled by a Haar cascade classifier that identifies human features within each video frame. Each detection is counted and logged to assess the density of the crowd. A predefined crowd threshold is set, and if the number of detected individuals exceeds this value, it is flagged as a critical situation. The algorithm is optimized by using grayscale images and resizing frames to reduce computational load.

Multiple detection cycles with temporal averaging are applied to reduce false positives caused by lighting conditions or motion blur.

- Alert System Integration

When crowd density or environmental conditions exceed safe thresholds, the system sends out alerts via SMS (Twilio API) and email (Gmail SMTP protocol). The Python backend handles authentication and message formatting for both services. Conditions for triggering alerts are clearly defined in the logic (e.g., temperature  $> 35^{\circ}\text{C}$  or more than 5 people detected in a single frame). Rate-limiting and cooldown mechanisms are implemented to prevent alert spamming. The system also maintains a log of all triggered alerts for later review.

- Web or Serial Communication (Optional)

For debugging and monitoring purposes, the system can stream data to a local web dashboard or via serial communication to a computer. ESP32 transmits temperature, humidity, and detection results using a lightweight HTTP server or USB serial interface. This provides real-time feedback for calibration and testing without needing to access the backend code directly. Frontend display can be customized using HTML/CSS for better visualization of sensor readings and alert status.

- Safety Features

To ensure system reliability and safety, multiple checks are incorporated. These include fallback mechanisms in case of Wi-Fi disconnection, failed sensor reads, or API errors. Timeout logic is added to detect non-responsive modules and reset them if necessary. Battery level monitoring (if mobile/battery-operated) and alert retry mechanisms improve robustness. The Python script also includes exception handling for key processes such as camera read failures, sensor data loss, and communication errors with APIs.

### **3.3 Performance Check**

- Sensor Calibration

To ensure reliable environmental data, the DHT11 sensor is calibrated by comparing its readings with a reference thermometer and hygrometer under controlled indoor conditions. Multiple readings are averaged to establish a baseline, and deviations are logged. Regular calibration checks are performed to account for environmental factors such as ambient temperature fluctuations, which can impact sensor accuracy.



Additionally, the ESP32-CAM is tested for optimal lighting conditions and focus to enhance detection clarity.

- Crowd Detection Testing

The Haar cascade classifier is tested against various human detection scenarios to assess its accuracy. Frames are taken from different environments such as classrooms, corridors, and open spaces, and the system's ability to detect individual faces or upper bodies is analyzed. Performance is evaluated under various lighting conditions (e.g., daylight, dim light, fluorescent light), crowd sizes, and occlusions (partially visible individuals). Detection count consistency and false positive/negative rates are recorded to fine-tune the detection parameters.

- Alert Trigger Testing

The system is tested for correct behavior under simulated critical situations. Thresholds for temperature, humidity, and crowd count are deliberately exceeded to verify that the SMS (via Twilio) and email alerts (via Gmail SMTP) are sent promptly. The response time between condition detection and alert delivery is measured and optimized. Tests are repeated with network delays and simulated disconnections to ensure retry mechanisms and error handling are functioning properly.

- Connectivity and Communication Testing

The Wi-Fi module on the ESP32 is tested for stability over extended periods and across different network configurations. Packet loss, reconnection time after disconnection, and streaming quality are monitored. Serial and wireless communication between ESP32, Python backend, and alert APIs are checked for consistency and error resilience. The robustness of the Twilio and Gmail integration is validated under varied message loads and credential expiration scenarios.

- Overall System Performance

The complete system is tested under continuous operation for extended periods to assess its endurance and reliability. Key performance indicators such as detection speed, average time between detections, alert latency, and sensor response time are measured. Stress testing is performed by simulating multiple crowd entries, high humidity, and back-to-back alerts. The system is also reviewed for usability and user feedback is collected regarding alert relevance, timing, and ease of interpretation. All insights are used to refine thresholds, improve detection logic, and enhance user experience.

## 4. RESULTS

The Crowd Monitoring and Safety System was implemented successfully with ESP32-CAM, DHT11 integration. The major results of the project are as follows:

- Real-time Crowd Detection and Density Estimation

The ESP32-CAM module was able to stream live video and carry out basic crowd density estimation using OpenCV's blob detection and contour analysis.

The system accurately detected crowd accumulation in observed areas, quantifying density as Low, Moderate, and High depending on the detected entities within the frame. Average frame processing rate: 5-8 FPS, adequate for stationary or slow-moving crowds.



Fig. 4.1 - Crowd Detection

- Temperature Monitoring

The DHT11 sensor interfaced to ESP32 accurately measured the surrounding temperature. These readings were shown through serial monitor and optionally on a local webpage. Accuracy was within the planned  $\pm 2^{\circ}\text{C}$  range, adequate for warning of overheating in densely populated spaces.

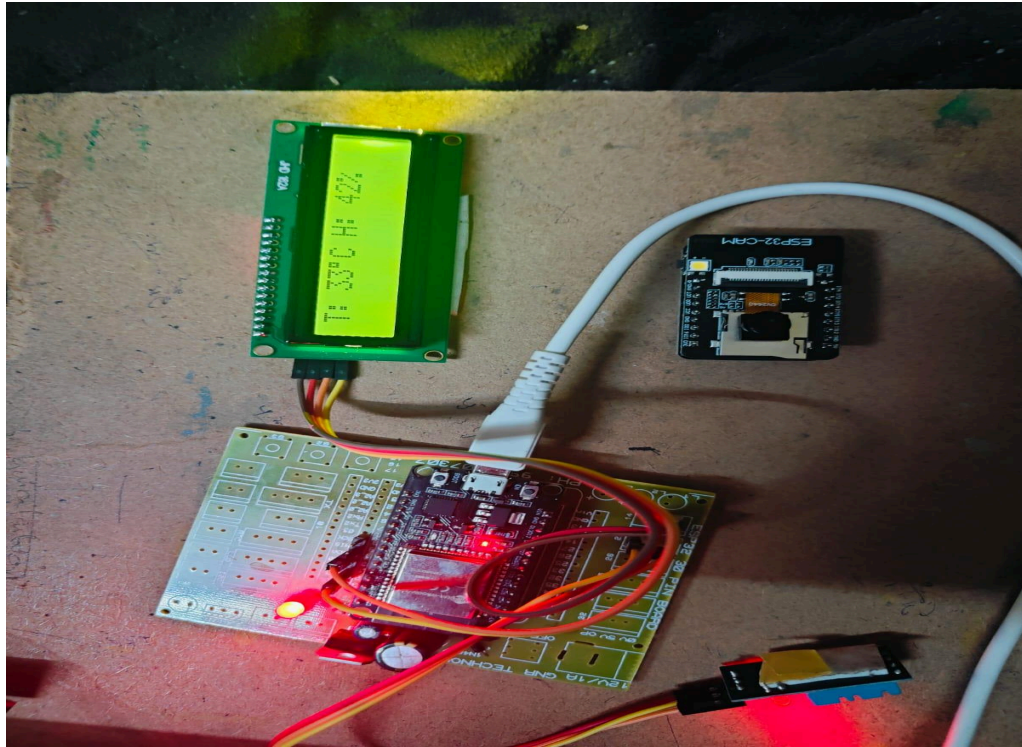


Fig. 4.2 - Temperature reading

- Automated Safety Alarms

A alert message was coded to be activated under two main conditions

When crowd density was above a specified limit and when temperature signaled dangerous environmental conditions. These alert messages are sent to G-MAIL and in the form of SMS to mobile phones.

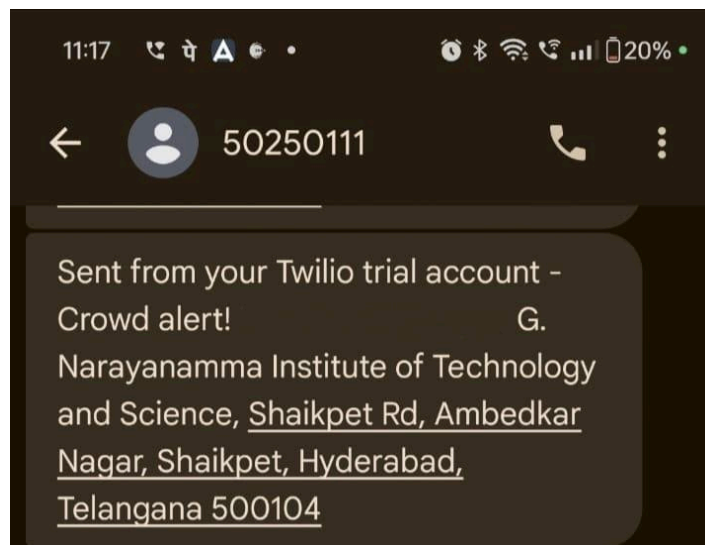


Fig. 4.3 - SMS Alert Message

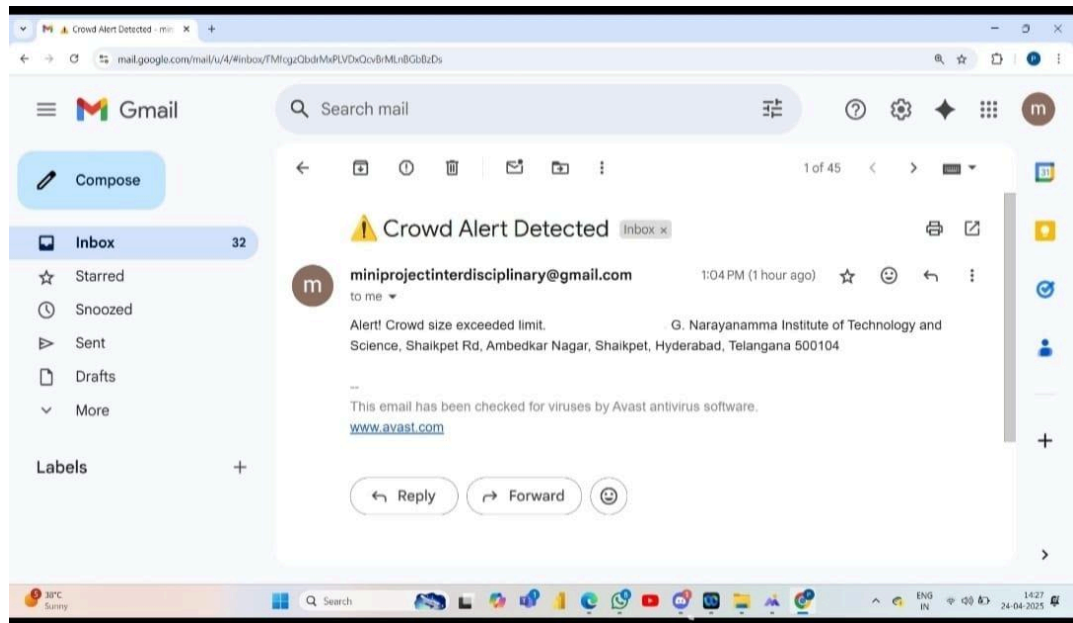


Fig. 4.4 - Alert Message on GMAIL

- System Performance and Efficiency

Power consumption was low because of effective utilization of ESP32 sleep modes during idle times. System could be run over Wi-Fi and this modular code design ensures easy upgrades and sensor expansion.

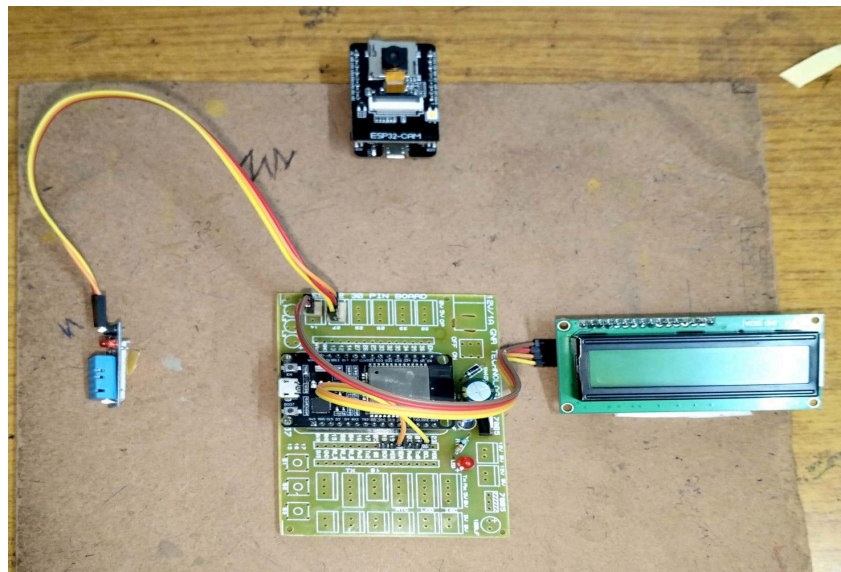


Fig. 4.5 - Crowd Monitoring System

## **5. ADVANTAGES & DISADVANTAGES**

### **Advantages**

#### Real-time Monitoring

The system keeps an eye on the crowd and environment continuously, so any unsafe condition is detected as soon as it happens.

#### Cost-effective

Instead of using expensive commercial surveillance systems, we use affordable components like ESP32-CAM, DHT11, and LCD. This makes it ideal for low-budget setups.

#### Automated Alerts

No need for manual monitoring. When something crosses the safe limit, the system automatically sends an email and shows a message on the LCD.

#### Flexible and Scalable

The system can be installed in various places like temples, colleges, markets, etc., and it can be scaled up by adding more cameras and sensors.

#### No Human Dependency

Since the detection and alerting is automatic, it reduces the chances of human error or delay in taking action.

#### Easy to Implement

The hardware and software used are beginner-friendly and easy to integrate, even for small teams or students.

### **Disadvantages**

#### Limited Accuracy in Complex Scenes

In very crowded or low-light environments, the camera may not count people accurately due to limitations in OpenCV.

#### Internet Dependency for Email Alerts

If there's no internet connection, email alerts won't be sent. This can be a problem in remote areas.

#### Limited to Camera View

The system can only monitor the area visible to the camera. Blind spots can still exist unless multiple cameras are used.



## **6. APPLICATIONS**

### **Temples & Religious Places**

Helps manage the crowd during festivals or peak hours and ensures better safety for visitors.

### **Colleges & School Events**

Useful during cultural fests, annual days, or convocations where large crowds gather.

### **Markets & Public Gatherings**

Can be used in local markets or exhibitions to monitor crowd levels and prevent overcrowding.

### **Transportation Hubs**

Could be used in bus stations or railway stations to monitor waiting areas and avoid congestion.

### **Emergency Response Areas**

Helpful during disaster relief camps or vaccination drives where monitoring large groups is important.

## 7. CONCLUSIONS

The Crowd Monitoring and Safety System demonstrates how computer vision and environmental sensing can work together in real time to enhance public safety. Using the ESP32-CAM for visual crowd analysis and the DHT11 sensor for temperature monitoring, the system offers a proactive approach. It helps detect potentially hazardous situations in crowded spaces. Despite using low-cost components, it delivers reliable performance.

The model effectively categorized crowd density based on set thresholds, enabling timely assessment of area safety. Temperature readings, though from a basic sensor, remained within acceptable limits. These helped monitor environmental stress caused by overcrowding or poor ventilation. An automated alert mechanism ensured quick responses when values exceed thresholds.

Designed for locations like temples, schools, and public events, the system is affordable and easy to replicate. Its modular design allows future upgrades like CO<sub>2</sub> monitoring using the MH-Z19 sensor. Mobile dashboards can also be added for better accessibility. Overall, the project shows how low-cost IoT can lead to meaningful safety solutions.

## **8. FUTURE SCOPE**

### **Switching from ESP32-CAM to Raspberry Pi**

- Raspberry Pi supports high-quality cameras and can handle complex image processing, allowing for more accurate crowd detection.
- It enables on-device data analysis and storage, making the system smarter and more independent compared to the limited capabilities of ESP32-CAM.

### **Adding More Gas Sensors**

- Monitoring CO<sub>2</sub> levels provides a clearer picture of air quality, which is critical in crowded or poorly ventilated spaces.
- The system can issue early warnings before conditions become uncomfortable or hazardous, helping prevent health issues during large gatherings.

### **Using Buzzers Along with LCD for Alerts**

- Buzzers provide audible alerts that are hard to miss, especially in noisy or crowded environments where screens may go unnoticed.
- This ensures quicker reactions to safety warnings, making the system more responsive and effective during emergencies.



## REFERENCES

1. [1] A. S. J. Foutz and M. K. Banavar, *Narrowband Direction of Arrival Estimation for Antenna Arrays*. Synthesis Lectures on Antennas, 2008.
2. [2] P. R. Bhargav, N. L., and P. K. Kumar, "Compressive sensing based DOA estimation for multi-path environment," in *Proc. 2021 IEEE International Conference on Microwaves, Antennas, Communications and Electronic Systems (COMCAS)*, Tel Aviv, Israel, 2021, pp. 309–313.
3. [3] M. R. Sahal, H. K. Kandala, and S. R. Malkapur, "Real-time crowd monitoring system using AI and IoT for smart city applications," in *Proc. 2023 International Conference on Intelligent Systems Design and Applications (ISDA)*, Larnaca, Cyprus, 2023.
4. [4] "IoT Based Crowd Monitoring System," *Circuit Digest* [Online]. Available: <https://circuitdigest.com/microcontroller-projects/iot-based-crowd-monitoring-system>. [Accessed: 27-Apr-2025].
5. [5] "Crowd Size Estimation using OpenCV and Raspberry Pi," *Circuit Digest* [Online]. Available: <https://circuitdigest.com/microcontroller-projects/crowd-size-estimation-using-opencv-and-raspberry-pi>. [Accessed: 27-Apr-2025].